

GeoWatch Interview Guide

Simple Concept & Architecture Revision Sheet

Page 1 of 2

1. WHAT IS GEOWATCH?

The Core Problem

During crowded events, emergency reports come from all over the area. Organizers get overwhelmed and cannot easily spot the actual danger zones amidst isolated or false alarms.

We need to answer: "Where are incidents clustering together right now, and which zones are becoming safety hazards?"

The Platform's Solution

GeoWatch acts as a live coordinator:

Collects GPS incidents → Finds nearby groups → Identifies risk zones → Updates a live map

30-Second Interview Answer

"GeoWatch is a real-time safety dashboard. It ingests geo-tagged incident reports from mobile users, runs validation and rate limiting on the backend, and groups them into risk hotspots using an optimized clustering algorithm. It then broadcasts these hotspots instantly over WebSockets to a live map interface for event organizers."

2. MAIN TECHNOLOGIES & ROLES

Technology	What it does in GeoWatch
Spring Boot	Powers the backend REST APIs, coordinate validation, scheduling, and WebSocket messaging.
PostgreSQL	Stores all incident data persistently and safely.
Hibernate / JPA	Acts as the bridge to save and load Java objects directly into PostgreSQL tables.
DBSCAN	The spatial algorithm that finds groups of nearby incidents to create hazard hotspots.
Grid Index	A custom optimization that speeds up clustering by checking only nearby map cells.
WebSockets (STOMP)	Sends real-time updates directly to monitoring dashboards without forcing page reloads.
React + Leaflet	Powers the web dashboard and renders live coordinates onto an interactive map.
k6	A load-testing tool used to simulate high user activity and measure API performance.
Railway & Cloudflare	Railway hosts the backend container and DB. Cloudflare Pages hosts the static React website.

3. HOW THE SYSTEM WORKS

User reports incident → Spring Boot API → Validate & Save → DBSCAN Groups points → WebSocket updates map

The Core Concept in Simple Steps:

- **Ingestion:** A user reports a safety concern. The backend validates if the report falls inside the event boundaries (geofencing) and rate-limits spam.
- **Clustering:** The server retrieves active incidents from the last 15 minutes, runs them through the DBSCAN algorithm, and determines risk levels (Low, Medium, High).
- **Push:** The server broadcasts the resulting cluster coordinates instantly over WebSockets. The dashboard receives it and re-renders hazard visual shapes.

4. BACKEND CODE STRUCTURE

Standard Layered Architecture

We separate concerns into logical layers:

Controller → Service → Repository → Entity

- **Controller** (`IncidentController`): The entry gate. Receives the incoming request, triggers checks, and sends back the incident ID.
- **Service** (`IncidentService`): The logic brain. Handles event timings, geofencing, rate limiting, and triggers DBSCAN.
- **Repository** (`IncidentRepository`): Database messenger. Asks PostgreSQL for matching incidents.
- **Entity** (`Incident`): Defines how an incident is structured inside the database.

DTOs (Data Transfer Objects)

DTOs carry data across the API separately from the database tables. This isolates client requests from database schema modifications.

- **Client sending a report:**
JSON HTTP Payload → DTO (`CreateIncidentRequest`) → Service → Entity → DB
- **Server sending cluster information:**
Service Runs DBSCAN → DTO (`ClusterResponse`) → JSON Broadcast

5. DBSCAN CLUSTERING: CONCEPTS & PARAMETERS

What is DBSCAN?

DBSCAN is a spatial algorithm that groups points close to each other. It finds natural clusters of any shape and ignores isolated points as random noise.

- **10 reports close together:** Forms a hotspot cluster (Risk Zone).
- **1 isolated report:** Treated as noise and does not trigger an alarm.
- **Why it fits:** We do not need to guess the number of clusters beforehand.

Parameters and Code Settings

The code (`DbscanClusteringService.java`) defines two thresholds:

- **Epsilon (`EPS = 50.0`):** 50 meters. The maximum distance between two incidents to count them as neighbors.
- **MinPoints (`MIN_PTS = 2`):** At least 2 incidents must be close to form a cluster. If fewer, they are treated as noise.

6. SPATIAL OPTIMIZATION: THE GRID INDEX

The Performance Bottleneck

Normally, finding neighbors requires comparing every incident with all other incidents on the map. This represents **$O(N^2)$ complexity** which slows down exponentially during heavy reporting load.

Our Solution: 2D Spatial Grid

We divide the map into 50m cells. Each incident is assigned to its cell. When searching for neighbors, we check only the host cell and its 8 adjacent cells (3x3 grid) rather than the whole database. This yields **$O(N)$ efficiency**.

0.92 ms: The measured DBSCAN execution time on active in-memory incidents in our benchmarks.

7. REAL-TIME WEBSOCKETS & BROADCASTING

Why WebSockets over Polling?

REST Polling: The browser repeatedly asks "Any updates?" every few seconds, wasting battery, database connections, and server bandwidth.

WebSockets: Establish a single, persistent two-way channel. The server pushes updates to the dashboard only when data actually changes.

STOMP Channels & Concurrency

STOMP organizes WebSocket streams into channels. The server broadcasts cluster updates to the topic `/topic/risk-updates/{eventId}`.

- **500+ Concurrent Clients:** The backend successfully handles 500+ long-lived connections open simultaneously in JVM memory.

8. SIMPLE DEPLOYMENT ARCHITECTURE

Cloudflare Pages (React Web App)



Railway Cloud (Spring Boot Backend)



Railway Hosted PostgreSQL DB

* Environment variables configure backend database connection secrets and frontend CORS allowed origins.

9. LOAD TESTING WITH K6

250 VUs

Simulated Virtual Users

88,000+

Total Requests Handled

732.99 /sec

Request Throughput

0%

HTTP Failure Rate

Basic testing concepts:

- **VUs (Virtual Users):** Simulated clients making back-to-back REST requests to mimic real load.
- **Throughput (Requests/sec):** How many API operations the Spring Boot engine successfully answered each second.
- **Error Rate:** The percentage of failed responses (which remained at 0% during our test).

The k6 Test Flow (`api_benchmark.js`)

For 60 seconds, 250 VUs constantly call three endpoints:

POST /api/incidents (33%) • GET /api/events/nearby (33%) • GET /api/admin/clusters/{eventId} (34%)

10. RAPID-FIRE INTERVIEW QA

1. Why DBSCAN?

It groups points by proximity, identifies clusters without a predefined count, and ignores isolated points as noise.

3. What is the grid index?

It divides the map into cells so DBSCAN only queries adjacent cells instead

2. Why not K-Means?

K-Means requires specifying cluster counts upfront, handles outliers poorly, and assumes spherical groups.

4. Explain the 0.92 ms speed.

It is the in-memory execution speed of DBSCAN algorithm over incidents,